I. Folder Description

remote_control-x86-can-v2

This folder contains the low-level driver code for the robot, including the drivers and control logic for two master arms and two follower arms. The system is modified from the cobo_magic platform and is divided into two parts: the **robot side** and the **teleoperation side**, although both share the same codebase.

Mechanical

This folder contains the mechanical drawings of the robot and the experimental platform, provided for reference.

script

This folder contains the main script programs for the project. See the following sections for detailed descriptions.

LLM_agent

This folder contains the LLM (Large Language Model) agent scripts used for data analysis.

Bag

This folder contains the robot's trajectories for running the demo of our experiment, including both the original and optimized trajectories. These can be directly called by the main program in the script to execute the motions on the robot.

II. System requirements

The system's hardware consists of two parts: The robot and teleoperation system, and the experimental workstation.

The requirements of robot and teleoperation system：

The robot and teleoperation system are modified from the cobo_magic platform from AgileX Robotics. The robot consists of a mobile base and two slave arms, with a recommended computing platform configuration of an Intel i7-12700H CPU and 16GB of memory. The robot is equipped, as needed, with customized grippers, pipettes, thermometers, spectrometers, and other devices.

The teleoperation system comprises a display and two master arms, with a recommended computing platform configuration of an AMD 5700U CPU and 16GB of memory.The software for both the robot and the teleoperation platform runs on Ubuntu 20.04 with ROS Noetic, and they communicate through a ROS master-slave configuration.

The requirements of experimental workstation：

The experimental workstation is equipped with devices such as heating plates，sensors，a cold well, supporting both manual and robot-operated experiments. The Mechanical folder contains mechanical diagrams of the aforementioned hardware systems for user reference.

The LLM agent component used for data analysis and processing here runs independently, and its installation and usage can be directly referenced from the contents of the LLM_agent folder.

III. Installation guide

The installation of the robotic arm control source code for the robot and teleoperation system follows a similar approach, as detailed below:

(1) Install Ubuntu 20.04 and ROS Noetic

(2) Install dependencies:

```
sudo apt install can-utils net-tools libkdl-parser-dev -y
sudo apt install libgflags-dev libgoogle-glog-dev libusb-1.0-0-dev libeigen3-dev
```

(3) Copy the source code into the project directory:

```
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws/src
git clone https://github.com/gl-jiang/EI-robot-control.git
```

(4) Compile:

```
cd ~/catkin_ws/src/EI-robot-control/remote_control-x86-can-v2/tools
sudo ./build.sh
```

(5) Configuration

• Robot is configured as the ROS Master, with IP address set to:
192.168.1.5 (example)

• Teleoperation is configured as the ROS Slave, with IP address set to:
192.168.1.7 (example)

Note1 : Ensure that all dependencies are properly installed and that the environment is correctly set up before compiling. After compilation, proceed with sourcing the workspace and launching the required ROS nodes as per the project documentation.

Note2: The program relies on a customized robotic system and its software and hardware configuration. The system is re-made based on the cobo_magic platform from AgileX Robotics (known as "松灵机器人"). Therefore, it may not run properly without the corresponding robotic system.Users can refer to the algorithms and the architecture of it to build their own systems for reproduction.

IV. Demo and Instruction for use

(A) Scrpit Program Dependencies

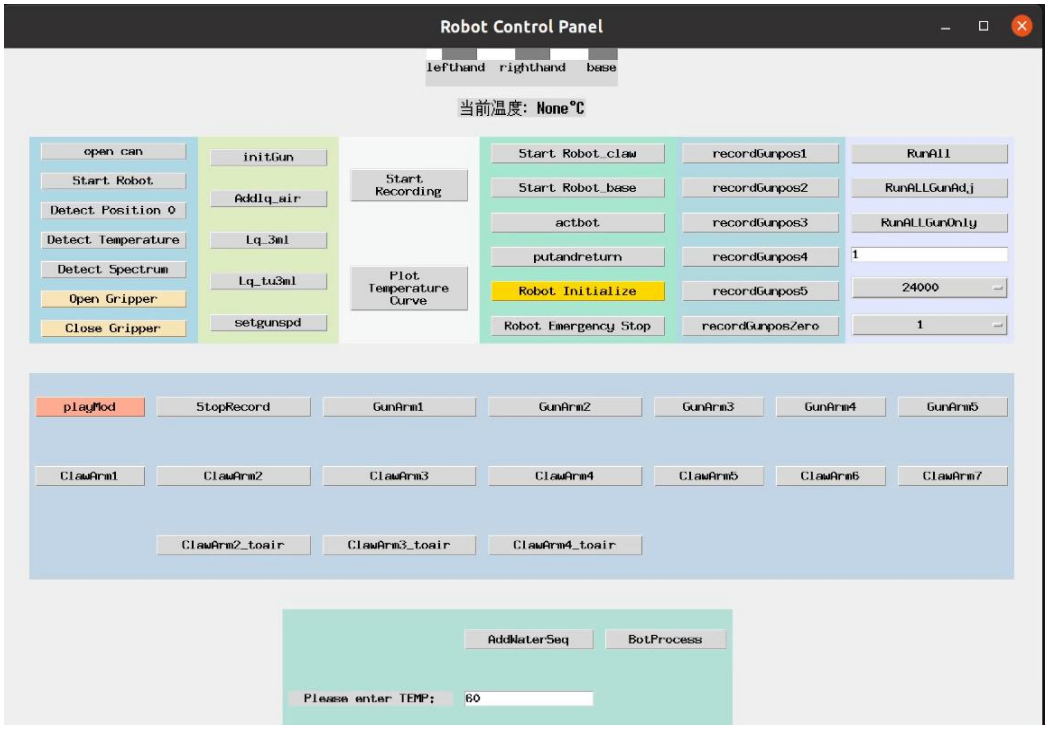
- Main Program: `actiongui.py` (Main interface program, used to start the robot, record trajectories, path points, and execute automated processes)
  - Dependency 1: `robot.py` (Robot startup node, gripper control, etc.)
    - Dependency 1.1: `claw.py` (Gripper control)
  - Dependency 2: `gun.py` (Robotic pipette control)
  - Dependency 3: `gui1.py` (Base class for the main interface program)
    - Dependency 3.1: `rosbagrecord.py` (Used for trajectory recording)
  - Dependency 4: `BLScontroller.py` (Tracking control)
  - Dependency 5: `GunPntRcord.py` (Records points into trajectory)
  - Dependency 6: `runGunPose2B.py` (Point-to-point control)

- Tool Programs: `ysmooth.py` (Trajectory optimization)

(B) Teaching (Operation)

1、Launch the program actiongui.py on the robot side.

The interface is described as follows:



After entering the above interface, the interface description is as follows:

The "Open Can" button in the top-left corner, when clicked, will execute the following procedure:

——

```
def open_can():
    print("Opening CAN")
    script_path = "../remote_control-x86-can-v2/tools/can.sh"
    gnome_command = ["gnome-terminal", "--", "bash", "-c", f"{script_path}; exec bash"]
    subprocess.Popen(gnome_command)
    print("CAN is openned")
```

——

——Its purpose is to execute the script ../remote_control-x86-can-v2/tools/can.sh, which is used to initiate the CAN connection for the robotic arm.

- The "Start Robot" button, located as the second button in the top-left corner, will execute the following procedure when clicked:

——

```
def start_robot():
    global process_robotstart
    print("Starting robot")
    script_path = "../remote_control-x86-can-v2/tools/jgl_2follower.sh"
```

```
gnome_command =   ["gnome-terminal", "--", "bash", "-c", f"{script_path}; exec bash"]
process_robotstart= subprocess.run(["bash", script_path])
time.sleep(2)
if process_robotstart is not None and process_robotstart.poll() is None:
    print("robot is started")
```

——Its purpose is to execute the script `../remote_control-x86-can-v2/tools/jgl_2follower.sh`, which is used to launch two follower robotic arms.

——Through the above steps, both robotic arms are started and ready to receive control commands.

2. Start the Program on the remote teleoperation Side

Execute the script `../remote_control-x86-can-v2/tools/jgl_Master.sh`, which contains the following content. Purpose: To start the two teleoperation master arms.

```
#!/bin/bash
workspace=$(pwd)
gnome-terminal -t "master1" -- bash -c "cd ${workspace}/master1; source devel/setup.bash && roslaunch arm_control arx5.launch; exec bash;"
sleep 1
gnome-terminal -t "master2" -- bash -c "cd ${workspace}/master2; source devel/setup.bash && roslaunch arm_control arx5.launch; exec bash;"
```

3. Teleoperation Recording

In the main interface of `actiongui.py`, the central area is used to record the teleoperated trajectory and actions of the robot. This area provides record/playback for 5 actions of the left gun_arm and 7 actions of the right claw_arm, respectively. It is recommended to decompose experimental operations into smaller segments for easier switching and combination.

For example, a liquid aspiration task can be decomposed into the following steps:

(1). Move from the initial position to the bottle mouth

(2). Insert into the bottle

(3). Aspirate and retract

(4). Move to the injection position

(5). Injection

(6). Return to the initial position

Among these, steps (1), (4), and (6) can be taught via human teleoperation. Steps (2) and (3)can use fixed motions based on inverse kinematics, such as vertical movement, to ensure accuracy and avoid collisions.

4. Trajectory Optimization and Control

After recording different actions of the experiment, the `ysmooth.py` script can be used for trajectory optimization. The optimized `rosbag` file can then be used directly for playback-based control.For actions requiring stability, the control method provided in `BLScontroller.py` (tracking

control) can be applied to suppress disturbances and improve robustness.
In the "bag" folder, there are recorded trajectory bags for quantum dot experiments as well as processed trajectory bags, which can be called upon for control execution.

5. Experimental Workflow

In `actiongui.py`, we provide two example implementations of experimental workflows:
- `RunAll_1217(self)` corresponds to a workflow using tracking control.
- `RunAll(self)` corresponds to a workflow using direct rosbag playback for control.

6. Results

For the results and videos, please refer to the supplementary materials of the paper.